

Esercizio 21 Casello

giovedì 14 maggio 2026 14:21

Un tratto di strada a senso unico è controllato due caselli X e Y. Ogni casello è dotato di una coppia di sensori, uno posto a 50 cm di altezza che rileva il passaggio di un'automobile e l'altro posto a 250 cm di altezza che rileva il passaggio degli autocarri. N.B. Quando passa un autocarro entrambi i sensori rilevano il passaggio. I sensori quindi sono in totale 4:

- per il casello X: SENS-X-B, SENS-X-A;
- per il casello Y: SENS-Y-B, SENS-Y-A.

Ogni volta che un mezzo attraversa un casello il relativo sensore basso invia un'interruzione allo z64. Una volta ricevuta l'interruzione da un sensore basso, lo z64 controlla se anche il relativo sensore alto ha rilevato un passaggio, interrogandolo direttamente. Lo z64 deve mantenere aggiornati quattro contatori globali a 32 bit:

- uno per il conteggio delle automobili entrate
- uno per il conteggio delle automobili uscite
- uno per il conteggio degli autocarri entrati
- uno per il conteggio degli autocarri usciti

I contatori vengono aggiornati in questo modo:

- i contatori delle automobili vengono incrementati ogni volta che un sensore basso rileva il passaggio, ma non lo rileva il relativo sensore alto
- i contatori degli autocarri vengono incrementati ogni volta che entrambi i sensori, alto e basso, rilevano il passaggio di un mezzo.

Al casello di uscita B è presente il dispositivo CASH che registra i pagamenti dei pedaggi. Ogni pedaggio pagato viene memorizzato, separatamente da tutti gli altri, in una memoria interna a CASH. Ogni importo di pedaggio è rappresentato da una parola di 16 bit. CASH ha anche un contatore a 32 bit che conteggia il numero di pedaggi pagati. Una periferica TIMER (che non deve essere programmata) invia ogni 8 ore un'interruzione allo z64. Quando lo z64 riceve questa interruzione programma un DMAC che trasferisce la lista dei pedaggi pagati in un buffer globale di memoria. Una volta trasferiti i dati CASH deve essere resettata (azzeramento contatore) e riavviata. Terminato il trasferimento dei dati, lo z64 deve determinare:

- Il totale per tutti i pedaggi riscossi da CASH e scriverlo in una variabile globale
(ATTENZIONE: se in memoria è già presente un totale questo deve essere aggiornato con i nuovi pedaggi riscossi. Il totale è un numero a 32 bit.)
- se c'è un qualche automezzo che è entrato e non ancora uscito dalla strada:
 - se c'è allora mette ad 1 un flag
 - se non c'è allora mette a 0 lo stesso flag

Si progettano:

- l'interfaccia della periferica CASH
- Il driver di gestione dell'interruzione di TIMER
- Il driver di gestione dell'interruzione di un sensore basso

ATTENZIONE: NON VA PROGETTATA L'INTERFACCIA DEL TIMER, LE INTERFACCIE DI TUTTI I SENSORI.

Soluzione

.org 0x800

.data:

```
.equ $sens_x_b_reg,0x0
.equ $sens_x_b_irq,0x1
.equ $sens_x_a_reg,0x2
.equ $sens_x_a_irq,0x3
.equ $sens_y_b_reg,0x4
.equ $sens_y_b_irq,0x5
.equ $sens_y_a_reg,0x6
.equ $sens_y_a_irq,0x7
.equ $dmac_reg,0x8 #usato per dmac
Auto_entrare: .word 0
Auto_uscite: .word 0
Autocarri_entrati: .word 0
Autocarri_usciti: .word 0
Pedaggio : .word 0 #importo pedaggio
Conta_pedaggi : .long 0 #contatori pedaggi
Tariffa : .word 5 # 5€ al km supposto
```

```

Km : .long 0 # contatore km
Totale : .long 0 #totale incasso
Buffer : .fill 28800 0 0 #calcolato in secondi
Entrato_non_uscito : . Word 0 # 0 tutti i veicoli usciti 1 almeno 1 veicolo dentro
.text
Main:
    Sti #abilito interruzioni
    .loop:
        Movl $0,%eax
        Inl $sensore_x_b_reg,%eax
        Cmpl $1,%eax
        Jc .vai
        Jnc .loop
    .vai:
        Addw $1,km
        Movl $1,%eax
        Outl %eax,sensore_x_b_irq #abilito interruzione
.driver 0: # sensore basso x
    Pushq %rax
    Pushq %rcx
    Pushq %r8 #macchina
    Pushq %r9 #tir
    Pushq %r10 #tariffa
    Movl $0,%eax
    Outl %eax,$sensore_b_x_irq #azzerato richiesta
    Call controlla
    Popq %r10
    Popq %r9
    Popq %r8
    Popq %rcx
    Popq %rax
    Iret
Controlla:
    #controllo macchine e tir
    Inl $sensore_x_a_reg,%eax
    Cmpl $1,%eax
    Jc .inc_tir
    Jnc .inc_macchina
.inc_tir:
    Addw $1,autocarri_entrati
    Jmp .esci
.inc_macchina:
    Addw $1,auto_entrare
    Jmp .esci
.esci:
    Inl %sensore_y_b_reg,%eax
    Outl %eax,$sensore_y_b_reg
    Movl $eax,%r8d
    Inl %sensore_y_a_reg,%eax
    Outl %eax,$sensore_y_a_reg
    Movl $eax,%r9d
    Cmpl $1,%r8d
    Jc .controlllo_tir
.controlllo_tir:
    Cmpl $1,%r9d
    Jc .inc_tir
    Jnc .inc_macchine
.inc_tir:
    Addl $1,autocarri_usciti
    Jmp .continua
.inc_macchine:
    Addl $1,auto_uscite
    Jmp .continua
.continua:
    #supponiamo pedaggio 5€ al km
    Movl $0,%ecx
    .loop1:
        Addw %r10d,tariffa
        Addw $1,eax
        Cmpw km,%eax
        Jnc .loop2
    Movw tariffa,pedaggio #pedaggio totale
    Addw $1,conta_pedaggi
    Ret

```

```

.driver1: #timer
    Pushq %rax
    Push %r11 #usato per vedere la differenza delle macchine
    Push %r12 #usato per vedere la differenza dei tir
    #levo azzittimento richiesta perché dmac
    Call manda
    Popq %r12
    Popq %r11
    Popq %rax
    Iret

Manda:
    #dmac
    Movw 28800, %eax
    Movw $buffer, %rsi
    Movw $dmac_reg, %dx
    Cld
    Outsb
.loop2:
    Addw totale, buffer(, %ecx, 4)
    Addw $1, %ecx
    Cmpw $28800, %eax
    Jnc .loop2
    Jc .controllo_mezzo_dentro
.controllo_mezzo_dentro:
    #controllo macchina
    Movw auto_entrare, %r11d
    Subw auto_uscite, %r11d
    #%r11d ha la differenza
    Cmpw $0, %r11d #macchine tutte uscite
    Jc .controlla_tir_usciti
.controlla_tir_usciti:
    #controllo macchina
    Movw autocarri_entrati, %r12d
    Subw autocarri_usciti, %r12d
    #%r12d ha la differenza
    Cmpw $0, %r11d #tir tutti usciti
    #vedo se c'è un mezzo almeno che non è uscito ancora
    Andw %r11d, %r12d #risultato in r12d
    Movw %r12d, entrato_non_uscito #scrivo nel registro
    Ret

```

Schema per cash(irq) :



